

Section 2: System Design Fundamentals

Client-Server Model

Client-server model is a fundamental concept in networking and communication that powers modern computing. Whether you are browsing the web, using an app, or streaming a video, you are interacting with this model everyday. Understanding how this model works is crucial for designing scalable and efficient systems.

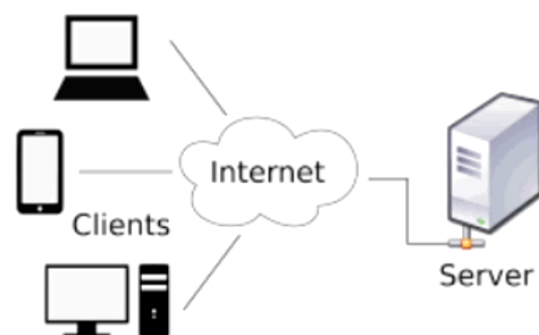
In this model, usually we have a client, such as your laptop, phone or even a smart device like a refrigerator. It sends a request to the server, which processes the request and sends back the response. This interaction happens millions of times per second across the internet. This enables everything from simple web pages to complex cloud-based applications.

In this session, we are going to break down the communication process, explore how the request-response cycle works, and discuss real-world applications of the client-server model. By the end, you will have a clear understanding of how this architecture plays a vital role in system design.

What is the Client-Server Model?

Client-server model is one of the most fundamental architectures in computing, and it serves as the backbone of modern applications. It doesn't matter whether the applications are web-based, mobile-based, or are cloud services.

The client-server mode is a computing framework where the clients request services and servers provide them. The client initiates communication while the server processes the request and responds accordingly. This separation of concerns allows for efficient resource management and scalability in our modern computing environments. It is the foundation of modern web, database, and application architectures.



Why is the Client-Server Model Important?

- Client-server model allows for efficient resource management, meaning computational power, storage, and processing optimally at the server level. Instead of each device handling complex processing, tasks are delegated to powerful servers.
- It enables seamless communication across various applications, whether you are accessing a website, you are sending an email, or you are interacting with an API. The functionality that you will get will be the same.

To understand the importance, let's look at some real-world examples. Imagine you are browsing a website. Your web-browser acts as the client. It sends a request to the web-server, and the server hosts that site. The server processes this request, retrieves the webpage, and sends back it to the browser for display.

Another example, we can think about is email communication. When you send an email from your computer, your email application functions as the client. It communicates with the email server to send, store and retrieve the messages. This client-server interaction ensures smooth and reliable email exchange across different devices and networks.

Therefore, as you can see, this model is everywhere, from websites to cloud computing to gaming to IoT devices. It is very crucial to understand this, and now that we understand what it really is and why it is important, let's move on to how the client-server communication actually works.

Key Components of the Client-Server Model

Let's break down this model into 3 main components, which work together to enable seamless communication in almost every digital service that we are using today.

- **Client:** This is the user-facing application that initiates the request. The client could be a web browser loading a web page, a mobile application, or an API consumer. The client wants to interact with the backend system. Clients are designed to request information, process the response that it is getting from the server, and display the results to the user.
- **Servers:** Servers are responsible for processing requests and returning the responses. Depending on the application, a server could be a web server that delivers web pages, a database server that retrieves and stores the information, or a mail server that manages the email communication between senders and receivers. Essentially, the server does the heavy lifting of all the functionality ensuring that the clients receive the correct data efficiently.

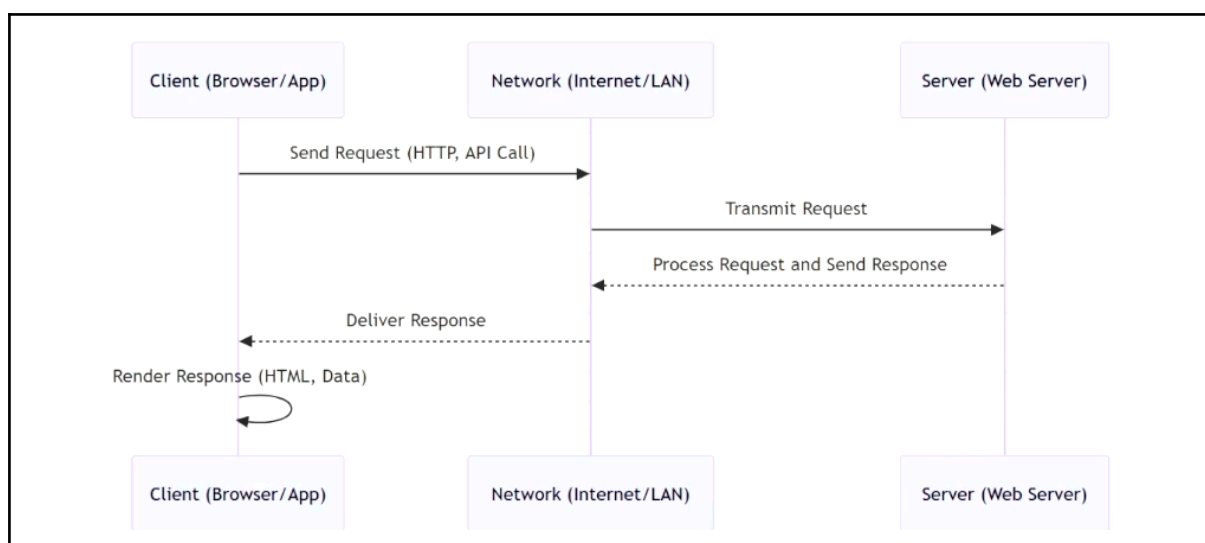
- **Network:** The invisible bridge connecting clients and servers are networks. Networks can take many forms, including the internet which powers global communication across the world, local area networks, which are used inside homes and businesses, connected through either wire or Wi-Fi connections, and even 5G networks which enables high-speed mobile data communication across mobile devices. Without a network, clients and servers would not be able to communicate at all.

In summary, the client sends requests, the server processes them and responds to those requests, and the network facilitates the connection between them. These 3 components form the foundation of the client-server model, ensuring smooth and scalable interaction in our modern applications.

How Clients and Servers Communicate

At a high level, the client-server communications follow a simple 5 step process:

1. The client sends a request. This could be an HTTP request for a web page or SQL query to a database or an API call from a mobile app.
2. The request is then transmitted over the network. It travels through the internet, a local area network or any other communication channel that lies between a client and a server.
3. The server receives and processes the request, and this could also involve retrieving data from the database, performing computations, calculations, and then finally generating a response.
4. Once the response is ready at the server, the server sends back the response. The response could be a web page, a JSON document, or a raw database result.
5. The response comes back to the client, the client receives and processes the response. It can then present the data, the response that it has received to the user. If it's a web page, it will display the web page, if it is raw data, it will update the UI based on the data that is received.



Not all client-server communication follows the same pattern. If we talk about the types of communication in the client-server model, there are mainly two types of communication in the client-server applications.

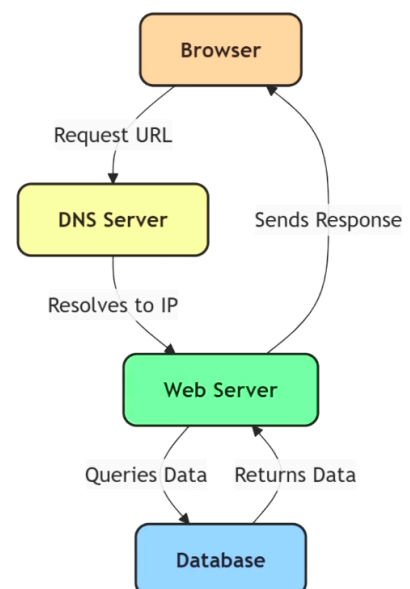
- **Request/Response Model:** This is the first type of client-server model. This is the most common type where the client sends a request and waits for the response before proceeding to do anything else. The main examples are: HTTP requests in web applications and the REST API calls that we make in microservice. This model is actually ideal for transactional interactions where each request is independent of one another.
- **Persistent Connections:** Unlike the above model, persistent connections keep the client and server continuously connected for real-time communication. The examples include web sockets, which enable live chat applications like whatsapp, FTP sessions, which enable file transfer across servers. All this happens because in this model, we maintain an open connection for continuous or large transfers of data over the network.

So whether it's a simple request-response interaction or a continuous real-time connection, client server communication forms the backbone of modern applications.

The Request-Response Cycle (HTTP Example)

The HTTP request-response cycle process happens every time you visit a web page, and if you understand it, it is very crucial for designing large scale systems.

1. It all starts when the browser requests a web page. Let's say you type example.com in your browser address bar and hit enter. But before the browser can reach the website, we first need to know where this website is located. That's where the DNS comes into picture.
2. The browser first needs to find the IP address of example.com. The DNS resolution will kick in. The domain name server translation happens at multiple levels. If the IP is cached for this domain, it is retrieved from the cache. If not, it will go to the domain name server and get the IP.
3. Once the browser has the correct IP address, it sends an HTTP GET request to the web server. This request is asking the server to provide the actual content of this web page. Now everything will happen on the server-side. The client-side part is done.



4. The web server receives the request and it will process it. If the page contains dynamic data, the server might need to query a database to fetch the data. It might also need some computations and calculations on top of the data. But when all the necessary information is gathered, the server will create a response.
5. The server will then send back the HTTP response. This response includes a status code like 200 if the response is successful or an error like 404 if the page that the user is requesting is not found and it sends back the actual content of the webpage which could be HTML, CSS, JavaScript or even raw JSON if it's an API response.
6. Finally, the browser receives the response and processes it. It renders the HTML, loads the HTML, loads the styles and JavaScript and displays the fully formed web page on the screen.

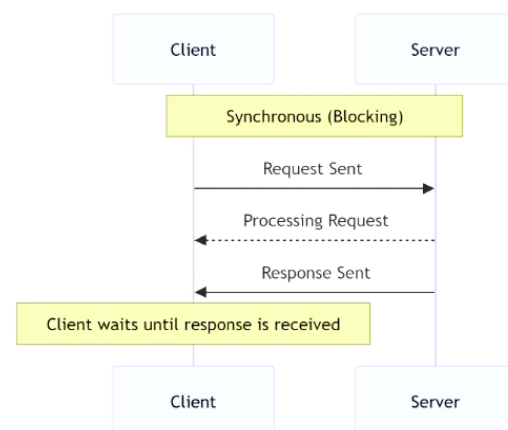
This entire request-response cycle happens just in milliseconds and it enables a smooth web experience that we are so reliant on nowadays.

Synchronous vs Asynchronous Communication

So far, we have discussed how client and server communicate, but not all interactions happen in the same way. Some applications require immediate response while others need to handle multiple tasks simultaneously and they might respond after some time, which brings us to the key difference between synchronous and asynchronous communication.

- **Synchronous Communication**

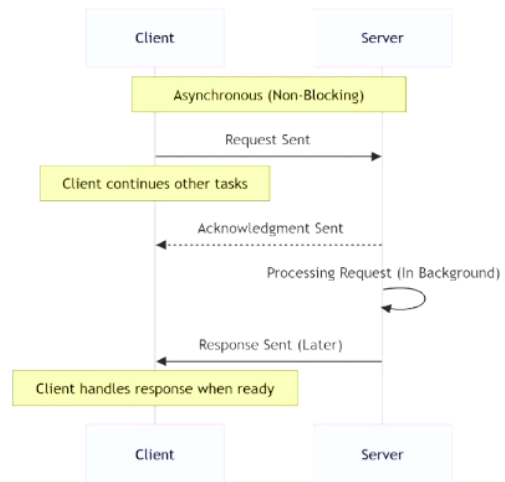
In the synchronous communication model, the client waits for a response before continuing. So it's a blocking call. The client makes a request and it waits for the response from the server before doing anything else. It follows a strict request-response pattern, meaning nothing else can happen until the server replies. A common example of this is submitting a form on a website. You fill out your details and you click submit. Then you wait for the confirmation message before doing anything else. This is typical working in the REST APIs and traditional web applications, where each request must be fully processed before moving to the next one.



However, this is not ideal, especially in case of real time-applications like WhatsApp, chat and all. That's where asynchronous communication comes in.

- **Asynchronous Communication:**

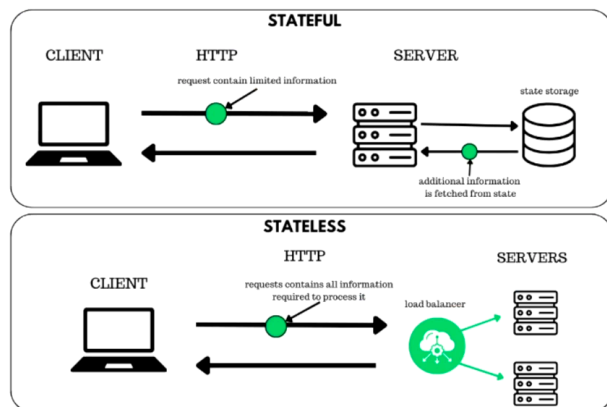
In the asynchronous communication model, the client does not wait for a response. Instead it can continue other tasks while the server processes the request in the background. Once the response is ready, the server will notify the client and then the client can get the response from the server. For example, we talked about real-time chat applications like WhatsApp. When you send a message in a chat application, the app doesn't freeze while waiting for a reply. Instead, it keeps running. And when the recipient responds, the new message appears instantly. This is powered by technologies like WebSocket, which enables continuous two-way communication and which allows all the web pages to fetch the data without reloading.



So two ways of communication, synchronous communication and asynchronous communication. If you have to summarize, synchronous communication is like making a phone call. You speak, wait for the response and then continue. Asynchronous communication, on the other hand, is like sending a text message. You send it and continue doing other things until you get a reply from the other side.

Stateful Vs Stateless

To understand the client-server interactions in more detail, there is another key distinction to understand, and that is stateless versus stateful servers. This difference determines how a server handles user interactions over time, and it also significantly impacts performance, scalability and user experience. This is why this is very important in terms of system design.



- **Stateless Server:**

These servers do not retain any memory of past interactions. So if I, as a user making a server, make multiple server calls, the stateless server will not retain any memory of the past interaction. Every request is treated as independent, meaning the server doesn't store any session details of the previous request. Each request must contain all the necessary information for the server to process it.

A very common example of this is REST API. when you make a request to the REST API, the server processes it and responds back, but it doesn't remember any previous interaction. Another example is basic HTTP web servers where you don't have to log in. Each request for a web page is handled separately without maintaining any user session data.

These servers are ideal for scalability because they can handle a large number of independent requests efficiently. They also enable better caching as the responses don't have to rely on the stored session data. Additionally, they work very well with load balancers, where we can distribute incoming requests across multiple servers, and it will ultimately improve the performance because none of the servers actually stores any of the user data.

However, many applications want to retain the user context. So that's where the stateful server comes in.

- **Stateful Server:**

Unlike stateless servers, stateful servers maintain session information across multiple requests. This means the server remembers previous interactions and keeps track of the user state throughout their session.

Some common examples of stateful systems include WebSockets, multiplayer online games where the server tracks player position as the game is progressing, banking applications where the user has to log in and the server has to maintain a session for processing the secure transactions.

The main benefit of a stateful server is personalization. They enable seamless user-specific experiences by remembering session details. This is essential for real-time applications like chat applications, gaming applications, banking applications, or any application where maintaining the user state and session is very critical. ‘

So to summarize, stateless servers are great for scalability and efficiency. They make the ideal choice for Web APIs and distributed systems. Stateful servers, on the other hand, enhance user experiences and personalization by maintaining session information. The choice between these two will depend on the requirement of the application

Real-World Examples

Now that we have covered the fundamentals of the client-server model, let's look at some real-world examples, where the architecture of the client-server model is used daily. Almost every digital interaction that we see nowadays relies on some form of client-server interactions.

- **Web Applications:** The web application traditionally and even nowadays relies on the client-server model for communication.
- **APIs & Microservices:** Web APIs and microservices, mobile applications, they also often communicate with backend services to get the data.
- **Databases:** In case of database servers, the web server acts as the client and the database server acts as the server. And whenever the application needs to retrieve or store the data, it will make a call to the database server and retrieve the information.
- **Messaging Systems:** WhatsApp, Slack, Messenger which uses web sockets to maintain a persistent connection between client and the server. This allows messages to be sent and received instantly and asynchronously.

So, you can pretty much see that client-server model is everywhere, from a simple web browsing to API-driven apps, to databases, to real-time messaging systems. And understanding these practical applications will actually help us in designing better systems in an efficient way.

Wrap up and What's Next...

- Key Takeaways:
 - The client-server model underpins most modern computing.
 - Different communication patterns (sync vs async, stateless vs stateful) optimize performance for different use cases.
 - Understanding this model helps in system design and scalability planning.
- Next Steps:
 - Forward Proxy Vs. Reverse Proxy

Interview Questions on the Client-Server Model

- Fundamental Questions:
 - What is the client-server model, and how does it work?
 - How does a client communicate with a server?
 - What are some real-world examples of the client-server model?
 - What is the difference between a client and a server?
- Request-Response Cycle Questions:
 - Explain the HTTP request-response cycle with an example.
 - What are the key differences between synchronous and asynchronous communication?
 - How does a browser load a webpage? Walk me through the steps.
- Architecture and Design Questions:
 - What is the difference between stateless and stateful servers?
 - How does caching improve performance in a client-server model?
 - How do load balancers work in a client-server architecture?
- Advanced Questions:
 - What are some security challenges in the client-server model?
 - How does WebSockets differ from traditional request-response communication?
 - What are some limitations of the client-server mode? How can they be addressed?
 - How would you design a scalable client-server system for a high traffic application?